



BILDUNG. FREUDE INKLUSIVE.

Webdesign im IT-L@B

Einführung in HTML und CSS
Positionierung mit CSS

Überarbeitet: 17.09.2024

von Dipl.-Ing. (FH) Robert Schoblick, M.Sc.



Kärntner Berufsförderungsinstitut GmbH
Bahnhofstraße 44
9020 Klagenfurt am Wörthersee
T. +43 (0)5 78 78
info@bfi-kaernten.at

www.bfi-kaernten.at



Die beste
Zeit für
Weiterbildung
ist JETZT!

Arbeitsauftrag:

1. **Lesen** Sie das Skript **vollständig**!
2. **Programmieren** Sie die vorgestellten Beispiele – auch in eigenen Varianten – nach. Vermeiden Sie es, fremde Programmskripte zu kopieren!
3. **Bearbeiten** Sie den am Ende der Lektion gestellten Arbeitsauftrag!
4. **Testen** Sie die Webseite auf verschiedenen Webbrowsern!

**Die Code-Beispiele sind als Grafik in dieses Dokument eingefügt.
D.h.: Copy&Paste ist keine mögliche Lösung!**

Arbeiten Sie NICHT mit ChatGPT oder anderen KI-Systemen!!!!

Kalkulierte Bearbeitungszeit

Die Aufgabe ist so gestaltet, dass Sie einschließlich der Lektüre dieser Lektion, dem Ausprobieren der Beispiele und eventuell eigene Recherchen im Internet maximal **6 Stunden Arbeitszeit** benötigen sollten. Wenn Sie Probleme (z.B. Schwierigkeiten mit dem Verständnis der deutschen Sprache) haben sollten, wenden Sie sich bitte an Ihre Trainerin oder Ihren Trainer.

Für die Abgabe der Aufgaben gelten folgende Regeln:

Offizielle Lehrlinge des IT-L@B:

Lehrlinge des IT-@B verfügen über einen Zugang zum Lernsystem Moodle. Rufen Sie den folgenden Kurs auf:

20E 55 - User Experience - IT-L@B Villach

Geben Sie die Arbeit im dafür vorgesehenen Abgabefeld der Aufgabe fristgerecht ab.

Praktikantinnen und Praktikanten:

Praktikantinnen und Praktikanten haben noch keinen Zugang zum Moodle-System. Sie geben die Aufgabe bitte in Ihrem **persönlichen Ordner** auf dem NAS („\\itlabvi\\public“) ab. Richten Sie dafür bitte ein Unterverzeichnis „web“ ein.

Inhalt

Arbeitsauftrag:	2
Kalkulierte Bearbeitungszeit	2
Für die Abgabe der Aufgaben gelten folgende Regeln:	2
Offizielle Lehrlinge des IT-L@B:.....	2
Praktikantinnen und Praktikanten:	2
Die CSS-Eigenschaft Position	4
Position: fixed;.....	4
Der HTML-Code (ein Beispiel):	4
Der CSS-Code (Beispiel):	4
Ergebnis der CSS-Regel „position: fixed;“:	5
Position: sticky;.....	5
Position: relative;	7
position: absolute;.....	8
Der HTML-Code für das Beispiel:.....	8
CSS-Code zu Abbildung 4:.....	8
CSS-Code zu Abbildung 5.....	9
CSS-Code zu Abbildung 6.....	11
position: static;	12
Die CSS-Eigenschaft z-index:	12
Achtung!	14
Weblinks zur Vertiefung (Auswahl):.....	15
Arbeitsaufträge.....	15

Die CSS-Eigenschaft Position

Die CSS-Eigenschaft **position** legt fest, wo ein Objekt in einer Webseite erscheint, in welchem Zusammenhang es zu anderen Elementen steht und wie es sich verhält, wenn sich die Dimensionen der gesamten Seite sowie deren Inhalte verändern (breiter/schmäler, scrollen etc.). Mithilfe der CSS-Eigenschaft **position** kann also erreicht werden, dass zum Beispiel ein Seitenkopf oder eine Menüleiste stets an der ihr zugewiesenen Stelle dargestellt wird.

Hierzu dienen zum Beispiel die Werte „fixed“ und „sticky“.

Darüber hinaus können Elemente auch *relativ* zu ihrer regulären Erscheinungsposition im Fluss der Seite dargestellt werden (*position: relative;*). An einer bestimmten, klar definierten Position in der Seite oder innerhalb eines Containers kann ein Element *absolut* positioniert werden.

Diese vier Positionierungen sollen Thema dieser Lektion sein.

Die CSS-Eigenschaft position gehört zu den wichtigsten CSS-Formaten im Webdesign!

Position: fixed;

In dem folgenden Beispiel soll ein Bild fest im Browserfenster verankert werden. Es spielt dabei keine Rolle, wie lang der übrige Inhalt ist. Der nachfolgende HTML-Beispielcode deutet bereits an (...), dass der Inhalt sehr umfangreich sein kann.

Es spielt bei dieser Form der Positionierung auch keine Rolle, an welcher Stelle im HTML-Code das zu fixierende Objekt geschrieben wird. Ausschlaggebend sind die festgelegten Koordinaten im Browserfenster.

Probieren Sie es einmal aus. Die Bilddatei liegt im Verzeichnis des Moduls. Schreiben Sie eine HTML-Datei, die beispielsweise diese Struktur hat.

Der HTML-Code (ein Beispiel):

```
<body>
...
<h1>Feste Platzierungen mit "position: fixed;"</h1>
<div>
<h1>Beispielinhalt</h1>
<p>Der "article"-Bereich ... platziert.</p>

<p>Unterhalb ... Randbereich.</p>
<p>Bei weniger ... bestimmt.</p>
<p>Dies ist ein Fließtext ... wird.</p>
</div>
...
</body>
```

Der CSS-Code (Beispiel):

Zum o.g. HTML-Code passt der nachfolgende CSS-Code. Theoretisch kann auf die Formate des <div>-Containers verzichtet werden. Experimentieren Sie etwas mit verschiedenen Szenarien. Verändern Sie dabei auch die Koordinaten „left“ und „top“.

```
div {  
  
border: 2px solid green;  
  
margin-left: 50px;  
  
}  
  
img {  
  
width: 80px;  
  
position: fixed;  
  
left: 50px;  
  
top: 50px;  
  
}
```

Ergebnis der CSS-Regel „position: fixed;“:

Das Bild des Hundes bleibt immer an der festgelegten Position in der Seite. Hier: Abstand vom linken Rand: 50px, Abstand von oben: ebenfalls 50px. Auch wenn der Text gescrollt wird, bleibt das Bild an dieser Position.

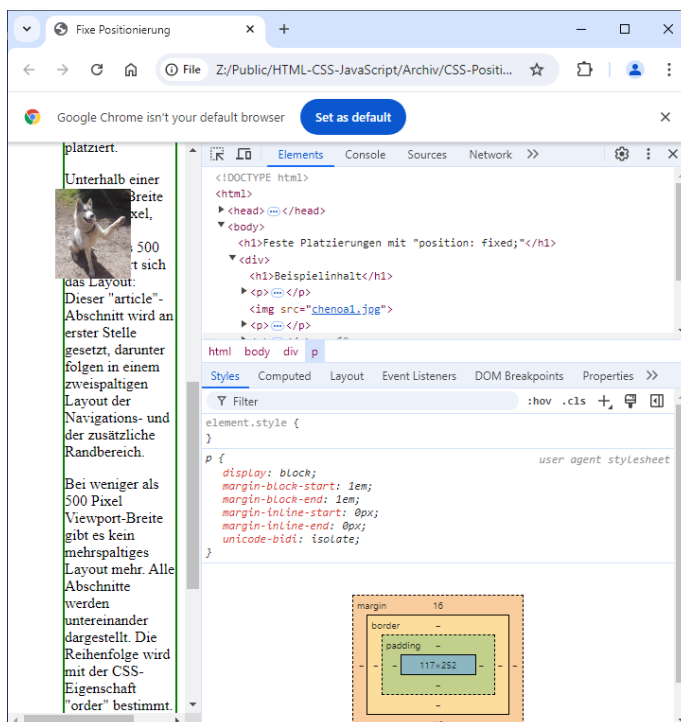


Abbildung 1: Das Bild bleibt auch beim Scrollen des Textes fest auf der definierten Position (position: fixed;)

Position: sticky;

Die Regel **position: sticky;** ist der *Fixierung* ähnlich, jedoch bewegt sich das Element innerhalb der am oberen und unteren Rand festgelegten Grenzen mit dem regulären Textfluss. Erst beim Erreichen der Grenzen wird das Element vom Textfluss entkoppelt und bleibt an dieser Stelle stehen, während die übrigen Inhalte über die Seitengrenzen hinweg scrollen.

Der folgende HTML- und der dazu passende CSS-Code demonstrieren anhand eines Beispiels das Prinzip. Die Abbildungen zeigen, wie sich das Element während des Scrollens verhält:

In Abbildung 1 bewegt sich das Bild wegen der Fixierung überhaupt nicht, auch dann nicht, wenn der Text sehr lang ist und der Inhalt im Fenster gescrollt wird. Anders sieht es dagegen in der Abbildung 2 aus. Das Bild in der Webseite folgt beim Scrollen dem Textfluss, jedoch nur bis es die definierten Grenzen erreicht. Sobald dies passiert, verharret das Bild auch in diesem Fall – vergleichbar der Fixierung – an der erreichten Stelle. Wird der Text wieder in die entgegengesetzte Richtung gescrollt, ordnet sich das Bild wieder an seiner ursprünglichen Position ein und folgt dem Textfluss.

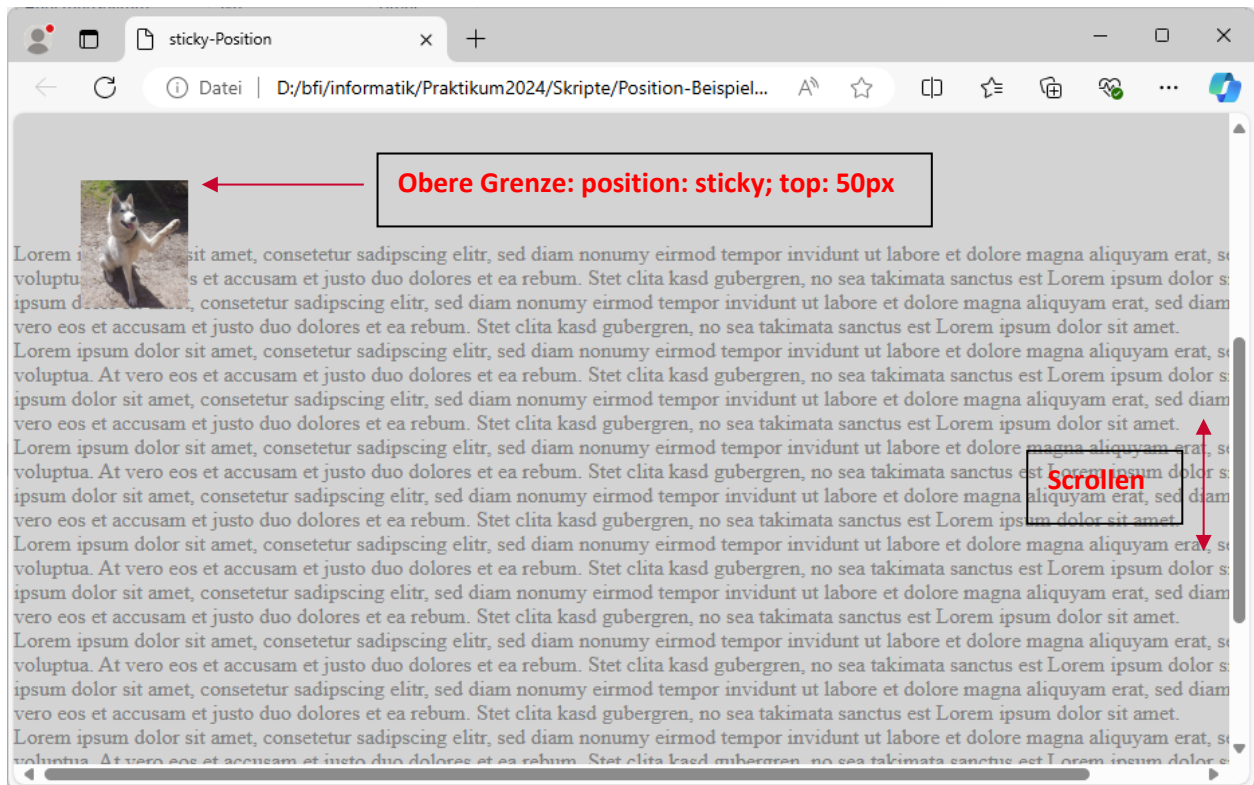


Abbildung 2: Das Bild bewegt sich mit dem Scrollen innerhalb der festgelegten Grenzen zum Rand des Viewports mit dem Textfluss. Beim Erreichen der Grenzen bleibt das Bild stehen und nur der Text bewegt sich mit dem Scrollen weiter.

Beispiel für einen HTML-Code (vgl. Abbildung 2, nur das `<img>`-Element wird in der Position bearbeitet):

```
<body>
<h1>Beispiel für "position: sticky;"</h1>
<main>
  <p>Lorem ipsum dolor sit amet, consetetur sadipscing elitr, ...</p>
  
  <p>Lorem ipsum dolor sit amet, consetetur sadipscing elitr, ...</p>
</main>
</body>
```

Der dazu relevante CSS-Code zeigt die obere (top) und untere (bottom) Grenzen. Das Verhalten des Bildes beim Scrollen wird mit der Regel „*position: sticky*“ festgelegt.

```
img {  
  width: 80px;  
  position: sticky;  
  top: 50px;  
  bottom: 20px;  
  left: 50px;  
}
```

Merke: *position: sticky*; kann nur funktionieren, wenn mit den Eigenschaften top und/oder bottom Grenzen festgelegt werden!

Position: relative;

Spricht man von einer *relativen* oder *absoluten* Positionierung, so ist zunächst einmal zu klären, von welcher *Ursprungsposition* ausgegangen wird. Bei der *relativen Positionierung* ist dies der Ort, an dem das betroffene Element normalerweise in der Webseite dargestellt wird.

Merke: *Relativ* bedeutet: Bezug auf den aktuellen und individuellen Standort. Bei einem HTML-Element folgt dieser „Standort“ also stets auch dem Textfluss in der Seite.

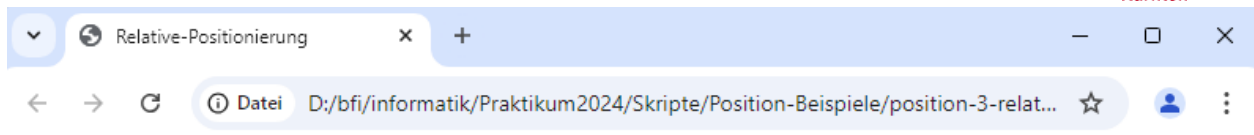
Die Abbildung 3 zeigt ein Beispiel: Hier ist ein Text (in einem -Element eingefasst) mit gelbem Hintergrund zu sehen. Das Element wurde mithilfe der folgenden CSS-Regeln formatiert:

```
span {  
  position: relative;  
  color: blue;  
  background-color: yellow;  
  left: -100px;  
  top: 50px;  
}
```

Wichtig ist die Regel *position: relative*; Diese wird durch konkrete Positionierungsangaben mit den CSS-Eigenschaften *left* und *top* die Positionierung ergänzt. Erst dadurch erfolgt eine Verschiebung des Elements. Im Bild zeigt das gestrichelte Rechteck den Ort, an dem das Element normalerweise im Textfluss erscheinen würde. Die Pfeilspitze zeigt auf den Ort, an dem das Element – ausgehend von dem (gestrichelten) Ort durch *position: relative*; verschoben wurde. Konkret erfolgt die Verschiebung um 50px nach unten (positiver Wert für „top“) und um 100px nach links (negativer Wert für „left“).

Die CSS-Regel *position: relative*; hat – abgesehen von der sichtbaren Verschiebung mit *left*, *right*, *top* oder *bottom* – aber noch weitere Wirkungen:

- Der ursprünglich vom verschobenen Element benötigte Platz wird NICHT freigegeben! Die Abbildung 3 zeigt deutlich den – durch das gestrichelte Rechteck gekennzeichneten – leeren Raum im Absatz.
- Sie setzt für die nachfolgend beschriebene Regel *position: absolute*; den „Nullpunkt“ bzw. die Ursprungsordinate.



Platzierungen mit position

Dies ist ein Fließtext innerhalb eines Absatz-Elementes. In diesen Absatz ist ein Textabschnitt eingefügt, der relativ platziert werden kann.

`left: -100px; top: 50px;`

Abbildung 3: Bei der relativen Positionierung wird das betroffene Element - ausgehend von der ursprünglichen Position im Textfluss - in der X- und/oder Y-Achse verschoben. Der ursprünglich benötigte Platz im Textfluss wird jedoch nicht freigegeben!

position: absolute;

Die absolute oder auch „freie“ Positionierung ermöglicht die gezielte Darstellung eines Elements an einer bestimmten Position auf dem Bildschirm oder innerhalb eines Container-Elements. Grundsätzlich markiert der linke obere Bildschirmpunkt den „Nullpunkt“, von dem ausgehend die absolute Positionierung stattfindet. Dies lässt sich allerdings auch ändern.

Das Prinzip der absoluten Positionierung wird in den folgenden drei Beispielen verdeutlicht. Alle Beispiele basieren auf dem gleichen HTML-Code.

Der HTML-Code für das Beispiel:

```
<div>
  <p>Dies ist ein Fließtext innerhalb eines Absatz-Elementes.  In diesen Absatz ist ein Bild - ein Inline-
  Element - eingefügt, das nun absolut, also ausgehend vom oberen linken
  Punkt des Absatzes platziert wird.</p>
</div>
</body>
```

CSS-Code zu Abbildung 4:

Im ersten Beispiel zeigt der CSS-Code zunächst keine für die Positionierung relevanten Formate. Es wird zunächst ein `<div>`-Container (grün gekennzeichnet) mit einem Rahmen umgeben. Dessen durchgezogene Linie hat eine Dicke von 2px und in der Webseite eine grüne Farbe.

Der innerhalb des zuvor beschriebenen `<div>`-Containers eingebaute Absatz (HTML-Element `<p>`) wird mit den rot hervorgehobenen CSS-Zeilen formatiert. Auch hier wird lediglich eine rote Umrahmung vorgesehen, die eine Breite von 2px haben soll.

Die Umrahmungen, welche beide Elemente betreffen, sind in Abbildung 4 bis Abbildung 6 deutlich zu erkennen. Beachten Sie in den folgenden Abschnitten den Abstand des Bildes zur linken oberen Ecke des Viewports sowie zu den linken oberen Ecken dieser Rahmen!


```
div {
  border: 2px solid green;
  margin-left: 100px;
}
p {
  border: 2px solid red;
}
img {
  position: absolute;
  left: 50px;
  top: 50px;
}
```

Interessant sind nun die blau gekennzeichnete Zeilen im CSS-Code: Die Regel *position: absolute;* legt fest, dass es sich um eine absolute Positionierung des `<img>`-Elements handelt. Die Koordinaten der Position werden (hier) mit den CSS-Eigenschaften *left* und *top* festgelegt. Möglich ist auch die Positionierung vom unteren Bereich (CSS-Eigenschaft: *bottom*) oder vom rechten Rand (CSS-Eigenschaft: *right*) ausgehend.

Positioniert wird nur das Bild (HTML-Element `<img>`). Dieses ist Inhalt eines Absatzes. Es ist also weder am Beginn noch am Ende des Absatzes in das `<p>`-Element eingebunden. Trotzdem funktioniert die freie Positionierung.

Wichtig: Anders als bei der relativen Positionierung wird der ursprünglich vom positionierten Element benötigte Platz in der Seite nicht leer belassen! Der Textfluss rückt bei der absoluten Positionierung in den durch die Verschiebung des Elements frei gewordenen Bereich nach!



Abbildung 4: Hier ist der linke obere Punkt im Fenster des Browsers der Ursprung für die absolute Positionierung.

CSS-Code zu Abbildung 5

Der nachfolgende CSS-Code zeigt in den grauen Code-Zeilen exakt den gleichen Inhalt wie er beim Beispiel in Abbildung 4 verwendet wurde. Lediglich eine Zeile wurde im CSS-Code für den `<div>`-

Container ergänzt (blau hervorgehoben):

```
div {
    position: relative;
    border: 2px solid green;
    margin-left: 100px;
}
p {
    border: 2px solid red;
}
img {
    position: absolute;
    left: 50px;
    top: 50px;
}
```



Abbildung 5: In diesem Bild wurde der `<div>`-Container mit **position: relative;** als Ursprung für die absolute Positionierung der darin enthaltenen Elemente festgelegt.

Die blaue Code-Zeile legt fest, dass der `<div>`-Container relativ positioniert werden soll. Da jedoch keine Koordinaten (Festlegung durch `top`, `bottom`, `left` oder `right`) definiert wurden, verbleibt der `<div>`-Container an seiner Position im Textfluss. Es erscheint also zunächst sinnlos, an dieser Stelle eine relative Positionierung zu setzen.

Der Sinn wird ersichtlich, wenn man Abbildung 5 betrachtet. In diesem Bild wird die Schachfigur nicht mehr vom linken oberen Rand des Viewports ausgehend platziert, sondern es wird der grüne Rahmen (die Umrahmung des `<div>`-Containers als Referenz für die Positionierung verwendet. Die absolute Positionierung des Bildes bezieht sich also auf den `<div>`-Container, nicht mehr auf die gesamte Seite!

Experimentier-Auftrag: Probieren Sie für die absolute Positionierung (Format des `<img>`-Containers) auch die CSS-Eigenschaften `bottom` und `right` aus. Versuchen Sie auch, negative Koordinaten zu verwenden.

CSS-Code zu Abbildung 6

Noch einmal erscheint der CSS-Code des Beispiels nahezu identisch zu den vorangegangenen Formaten. Lediglich die blau hervorgehobene Zeile wurde aus dem <div>-Container herausgenommen und stattdessen im Format des Absatz-Elements <p> eingefügt. Jetzt geben die Grenzen des Absatzes den „Nullpunkt“ für die Koordinaten zur absoluten Positionierung vor. Das Bild wird also in diesem Beispiel (top: 50px; und left: 50px;) vom linken oberen Eck des roten Rahmens ausgerichtet.

```
div {
    border: 2px solid green;
    margin-left: 100px;
}
p {
    border: 2px solid red;
    position: relative;

}
img {
    position: absolute;
    left: 50px;
    top: 50px;
}
```



Abbildung 6: Hier wurde der <p>-Container mit **position: relative;** als Ursprung für die absolute Positionierung der darin enthaltenen Elemente festgelegt. Selbst wenn der diesen Absatz umschließende <div>-Container mit der CSS-Regel **position: relative;** formatiert wurde, ist nun der linke obere Punkt des **Absatzes** der Ursprung für die Positionierung des darin enthaltenen Bildes.

Übungsauftrag: Versuchen Sie auch hier wieder andere Koordinaten, z.B.: mit den Eigenschaften right und bottom zu verwenden. Ändern Sie die Werte der Koordinaten und verwenden Sie auch einmal 0px für die beiden Richtungen. Was passiert, wenn Sie negative Werte verwenden?

position: static;

Ohne eine ausdrückliche Angabe einer Positionierung werden die Elemente so im Textfluss der Seite angeordnet, wie sie im HTML-Code programmiert werden. Auch hierfür gibt es einen Wert für die Eigenschaft position, der genau dies erzwingt: *position: static;*

Wird in einem CSS-Regelblock die Regel *position: static;* verwendet, haben Eigenschaften zur Definition von Koordinaten (*left, right, bottom, top*) keine Wirkung mehr.

Es sei noch einmal der HTML-Code auf Seite 8 und der dazugehörige CSS-Code (ebenfalls auf Seite 8) betrachtet. Würde man die Positionierung (im blau markierten Code-Abschnitt) entfernen, so würde genau das Bild im Browser erscheinen, wie es in Abbildung 7 zu sehen ist. Genau der gleiche Effekt wird durch die Regel *position: static;* erreicht. Hier kann gewährleistet werden, dass vorherige Positionierungen nicht auf das betreffende Element einwirken.

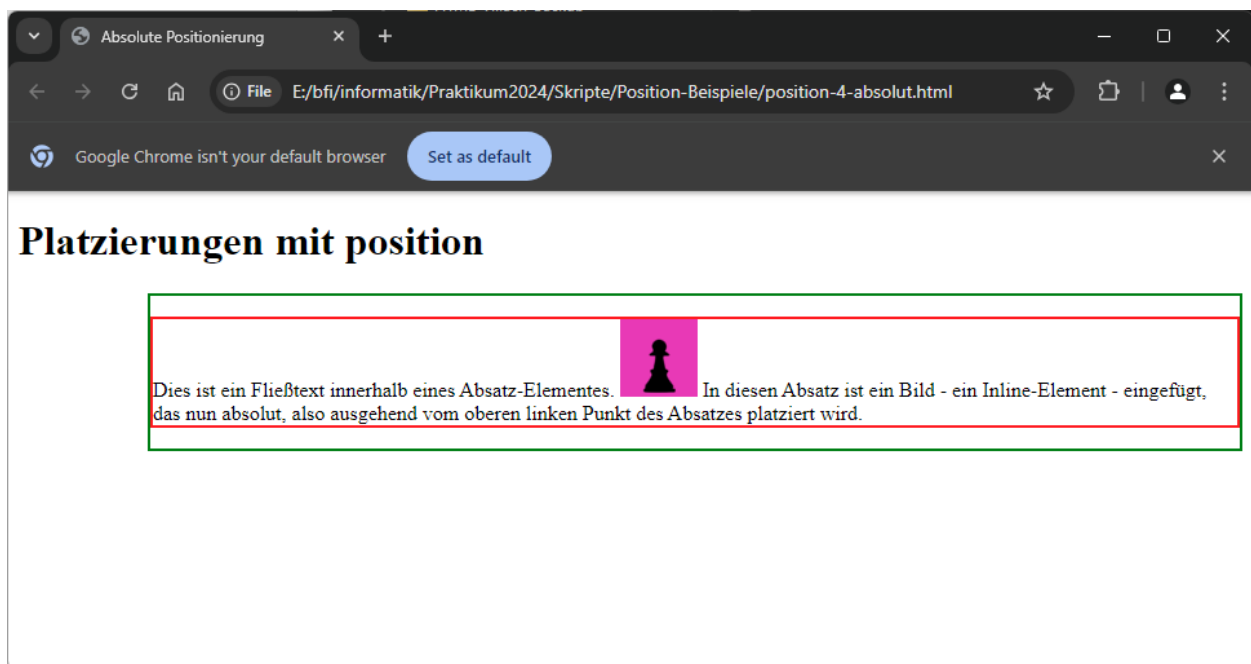


Abbildung 7: Mit *position: static;* wird das betroffene Element genau dort platziert, wo es in seinem regulären Textfluss erscheint. Diese Regel ist der Default-Wert. Sie wird nur dann verwendet, wenn sich vorangehende Positionierungen vererben würden.

Die CSS-Eigenschaft z-index:

Durch die Positionierung kann es passieren, dass sich verschiedene Elemente plötzlich überlagern und Informationen von anderen Elementen (z.B.: Text durch ein Bild) verdeckt werden. Kann man eine Überlagerung nicht vermeiden oder ist eine solche sogar gewollt, kann man mithilfe der CSS-Eigenschaft *z-index* die Reihenfolge festlegen. Mithilfe von *z-index* kann also bestimmt werden, welches Element „oben“ und welches Element hinter einem anderen Dargestellt wird. Grundsätzlich gilt, dass ein hoher Wert für *z-index* ein Element weiter nach vorn schiebt. Ein geringer Wert bedeutet, dass ein Element nach hinten verschoben wird.

Abbildung 8 zeigt zwei Bilder, wobei der Bauer (mit dem pinkfarbenen Hintergrund) den Turm überlagert und zum Teil verdeckt. Dem Bild-Element, welches den Bauern darstellt wurden folgende CSS-Zeile zugeschrieben:

Hinweis: Für die Illustration dieses Beispiels wurden in den `<img>`-Elementen Klassenattribute verwendet und den Klassen sinnvolle Namen zugewiesen. Das ändert aber nichts an der Vorgabe, dass SIE in Ihren Codes weitgehend auf die Verwendung von Klassen verzichten sollen, wenn als Alternative die Verwendung von semantischen HTML-Elementen möglich ist.

```
img.bauer {  
    position: absolute;  
    left: 50px;  
    top: 50px;  
    z-index: 2;  
}
```

```
img.turm {  
    position: absolute;  
    left: 75px;  
    top: 75px;  
    z-index: 1;  
}
```

Der Bauer hat in diesem Beispiel ein höheres z-index (2) gegenüber dem Turm (z-index: 1). Damit wird der Bauer über dem Turm dargestellt.



Abbildung 8: Dem HTML-Element, welches den Bauern auf pinkfarbenem Hintergrund darstellt, wurde das „z-index“ mit dem Wert 2 zugewiesen. Das Bild-Element mit dem Turm bekam das z-index 1 zugewiesen und erscheint deswegen hinter dem Bauern.

Nun sollen die z-Index-Werte vertauscht werden: Dem Bauer wird z-index: 1 zugewiesen, der Turm wird nun mit z-index: 2 formatiert. Probieren Sie es aus und beobachten Sie, was passiert. Vergleichen Sie das Ergebnis mit Abbildung 9!

```
img.bauer {
  position: absolute;
  left: 50px;
  top: 50px;
  z-index: 1;
}
```

```
img.turm {
  position: absolute;
  left: 75px;
  top: 75px;
  z-index: 2;
}
```

Jetzt hat der Turm in diesem Beispiel ein höheres z-index (2). Der Bauer wird nun hinter dem Turm dargestellt.

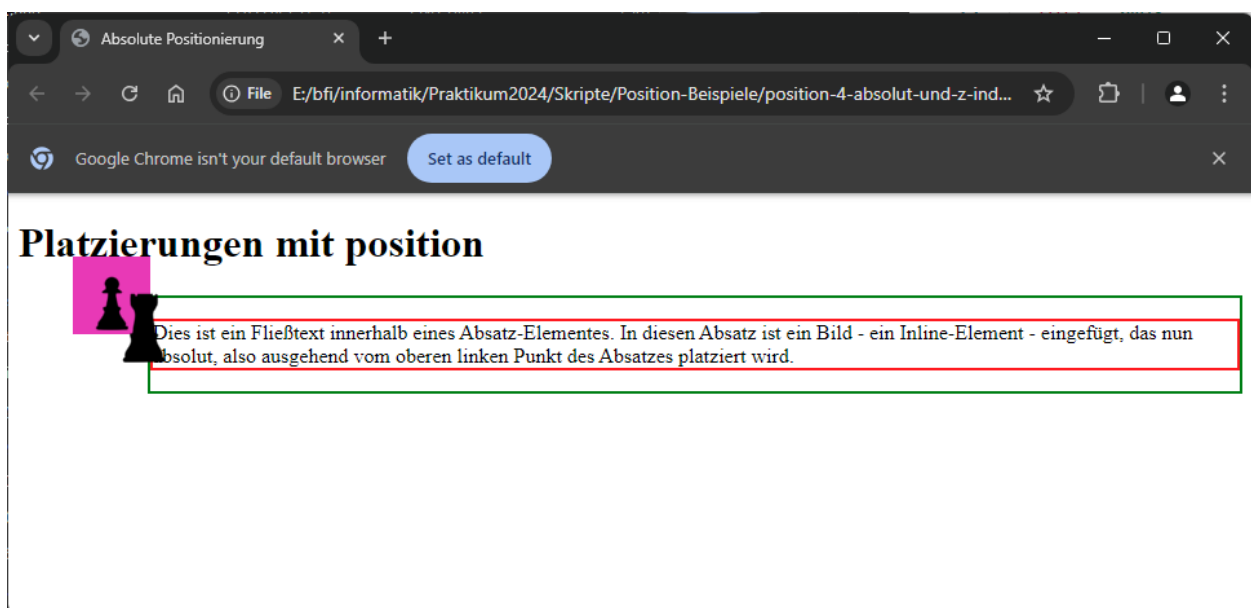


Abbildung 9: Der gleiche HTML-Code wie zuvor, jedoch wurden im CSS-Code die z-index-Werte beider Bilder vertauscht. Nun erscheint der Turm vor dem Bauern.

Wichtig: Mit der CSS-Eigenschaft z-index kann bewusst Einfluss auf die Reihenfolge einander überlagernder Elemente genommen werden. Dies kann wie gezeigt durch Positionierung passieren. Wird z-index nicht verwendet, gibt die Reihenfolge der Elemente im CSS-Code die Reihenfolge in der Überlagerung vor. Probieren Sie das einmal aus!

Achtung!

Die Positionierung mit CSS beeinflusst natürlich den Erscheinungsort des betroffenen Elements. Sie wirkt aber möglicherweise auch auf die Darstellung anderer Elemente ein. Dies kann zum Beispiel eine Überlagerung von Elementen sein, was im Endergebnis der Seite als nachteilig empfunden wird.

CSS-Positionierung gehört zu den wichtigsten CSS-Regeln. Insbesondere werden diese Eigenschaften für die saubere Gestaltung von HTML-Formularen – u.a. bei der Entwicklung der Benutzerschnittstelle eines Webshops – benötigt!

Weblinks zur Vertiefung (Auswahl):

https://www.w3schools.com/Css/css_positioning.asp

<https://wiki.selfhtml.org/wiki/CSS/Tutorials/Ausrichtung/position>

<https://www.anwahl.de/umfassender-leitfaden-fuer-css-positionierung-und-eigenschaften/>

Arbeitsaufträge

- Entwerfen Sie eine HTML-Seite mit voller Grundstruktur.
- Erstellen Sie im Body verschiedene Block-Container (an dieser Stelle **AUSNAHMSWEISE** <div>-Container) mit verschiedenen Klassen-Attributen. Es sollen mindestens 5 Container sein.
- Weisen Sie **jedem** Element eine Höhe und Breite in px zu (Achtung: Die Container sollen bequem gemeinsam auf dem Bildschirm Platz finden!)
- Weisen Sie **jedem** Container eine **eigene Hintergrundfarbe** zu. Sonst brauchen die Container keinen Inhalt.
- Ordnen Sie die Container mithilfe einer geeigneten Positionierung so an, dass ein Muster aus einander teilweise überlappenden Flächen entsteht.
- Verwenden Sie z-index, um die Reihenfolge bzw. die „Darstellungsebene“ der Flächen zu verändern.

Hinweis: Der CSS-Selektor auf ein Klassenattribut in einem HTML-Element beginnt mit einem Punkt!

Beispiel:

HTML:

```
<h1 class="test">....
```

CSS:

```
.test { }
```